

Java Backend Architecture

Interview Series

Chapter 11.8

Event Sourcing & Idempotency

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

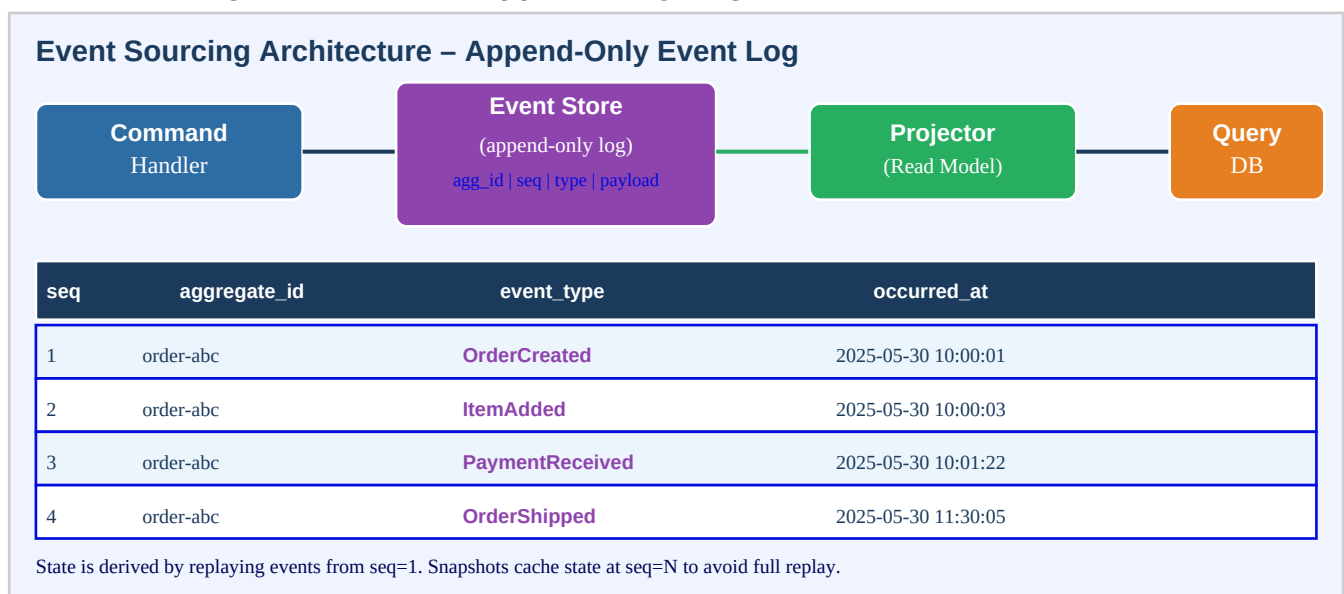
Chapter 11.8 – Event Sourcing & Idempotency

Introduction

Event Sourcing is an architectural pattern where the state of an aggregate is not stored directly as a mutable record, but is instead derived by replaying a sequence of immutable domain events stored in an append-only event log. Each event (e.g., `OrderCreated`, `ItemAdded`, `PaymentReceived`) represents a discrete fact about something that happened in the domain. The current state is computed by folding all events from the beginning of time (or from a snapshot). This provides a complete audit trail, enables time-travel queries, supports event-driven architectures, and decouples read models (projections) from the write model — making it a natural fit for CQRS (Command Query Responsibility Segregation).

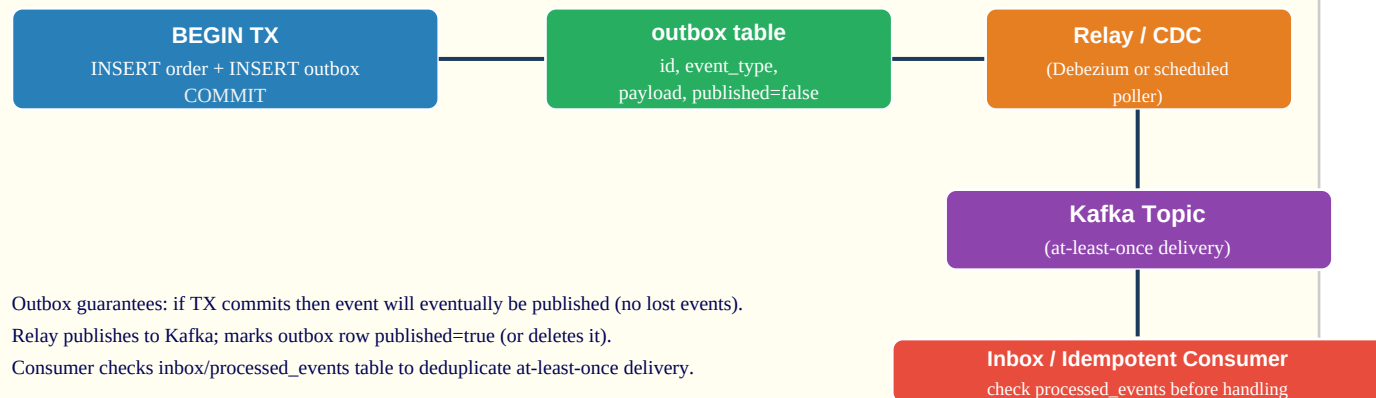
Idempotency is the property that executing an operation multiple times produces the same result as executing it once. In distributed systems, network failures, retries, and at-least-once delivery semantics make duplicate processing inevitable. Idempotency keys (client-generated UUIDs stored in a `processed_requests` table) allow servers to detect and deduplicate retried API calls. In messaging systems, the outbox pattern ensures exactly-once publication from a database transaction to a message broker, while inbox deduplication (checking a `processed_events` table before handling) prevents duplicate processing on the consumer side. Together, these patterns achieve effectively exactly-once semantics across distributed components.

Event Sourcing Architecture – Append-Only Log



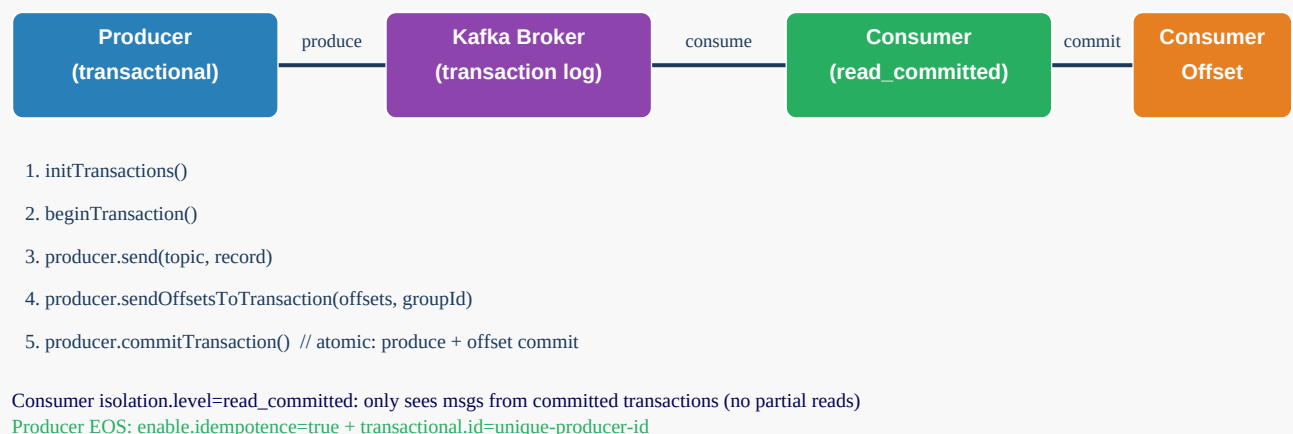
Transactional Outbox Pattern

Transactional Outbox Pattern – Atomic DB Write + Event Publish



Kafka Exactly-Once Semantics (EOS)

Kafka Transactions – Exactly-Once Semantics (EOS)



Key Definitions

Event Sourcing	Pattern where aggregate state is derived by replaying an immutable, append-only sequence of domain events; never mutate stored state
Event Store	Append-only database table or specialized store holding events: <code>aggregate_id</code> , <code>sequence</code> , <code>event_type</code> , <code>payload (JSON)</code> , <code>occurred_at</code>
Aggregate	DDD aggregate: a cluster of domain objects treated as a single unit for data consistency; events belong to one aggregate root
Sequence Number	Monotonically increasing integer per aggregate; used to enforce ordering and detect concurrent writes (optimistic concurrency on event log)
Projection / Read Model	A read-optimized view of the system state built by subscribing to events and updating a query DB; can be rebuilt by replaying the event log
Snapshot	Periodic checkpoint of aggregate state stored alongside event log; replay starts from snapshot + events after snapshot seq to avoid full log replay

Idempotency Key	Client-generated UUID sent with requests; server stores result in processed_requests table; duplicate requests return cached result without reprocessing
Outbox Pattern	Write domain event to outbox table in the same DB transaction as business data; relay process publishes outbox rows to message broker
Inbox Pattern	Consumer stores received event IDs in processed_events table before processing; duplicate event → skip (already processed)
Exactly-Once Semantics	Guarantee that each message is processed exactly once; achieved by combining at-least-once delivery + idempotent consumer
Kafka EOS	Kafka producer transactions: atomically publish messages + commit consumer offsets; consumer isolation.level=read_committed filters uncommitted
@TransactionalEventListener	Spring annotation: fires ApplicationEvent after the enclosing DB transaction commits (AFTER_COMMIT phase); ensures event not fired on rollback
CQRS	Command Query Responsibility Segregation: separate write model (commands, events) from read model (projections, queries)
CDC (Change Data Capture)	Mechanism (e.g., Debezium) that reads database WAL/binlog to capture row changes as events; used to implement outbox relay
Event Replay	Re-processing all stored events from beginning (or snapshot) to rebuild aggregate state or re-create a projection with new logic

Messaging Delivery Guarantees Comparison

Guarantee	Description	Duplicates?	Lost Messages?	Implementation
At-Most-Once	Fire and forget; no retry	No	Possible	UDP, basic HTTP no retry
At-Least-Once	Retry until acknowledged	Possible	No	Kafka default, SQS standard
Exactly-Once (EOS)	Atomic produce + offset commit	No	No	Kafka transactions + read_committed
Effectively Exactly-Once	At-least-once + idempotent consumer	Deduplicated	No	Outbox + inbox pattern

Code Examples

1. Event Store Schema and Append Logic

```
-- Event store table (PostgreSQL) CREATE TABLE domain_events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), aggregate_id VARCHAR(100) NOT NULL, aggregate_type VARCHAR(50) NOT NULL, sequence BIGINT NOT NULL, event_type VARCHAR(100) NOT NULL, payload JSONB NOT NULL, metadata JSONB, -- correlation_id, causation_id, user_id occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(), UNIQUE(aggregate_id, sequence) -- optimistic concurrency: prevents duplicate seq ); CREATE INDEX idx_events_aggregate ON domain_events(aggregate_id, sequence); CREATE INDEX idx_events_type ON domain_events(event_type, occurred_at); -- Snapshots table CREATE TABLE aggregate_snapshots ( aggregate_id VARCHAR(100) NOT NULL, aggregate_type VARCHAR(50) NOT NULL, sequence BIGINT NOT NULL, state JSONB NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), PRIMARY KEY(aggregate_id, sequence) ); -- Append event (Java/JDBC): // INSERT INTO domain_events(aggregate_id, aggregate_type, sequence, event_type, payload) // VALUES(?, ?, ?, ?, ?::jsonb) // Unique constraint on (aggregate_id, sequence) prevents concurrent appends at
```

```
same seq // → throws DataIntegrityViolationException → retry with incremented seq (optimistic concurrency)
```

2. Aggregate Rebuild via Event Replay with Snapshot

```
@Service public class OrderRepository { public Order load(String orderId) { // 1. Load latest snapshot (if any) Optional<AggregateSnapshot> snap = snapshotRepo.findLatestByAggregateId(orderId); Order order; long fromSeq; if (snap.isPresent()) { order = deserialize(snap.get().getState(), Order.class); fromSeq = snap.get().getSequence() + 1; } else { order = new Order(orderId); // empty aggregate fromSeq = 1; } // 2. Load events after snapshot List<DomainEvent> events = eventStore.loadEvents(orderId, fromSeq); // SELECT ... WHERE aggregate_id=? AND sequence>=? if (events.isEmpty() && snap.isEmpty()) { throw new AggregateNotFoundException(orderId); } // 3. Replay events to rebuild state for (DomainEvent event : events) { order.apply(event); // order.apply() is a pure function, no side effects } return order; } public void save(Order order, List<DomainEvent> newEvents) { // Append each new event; UNIQUE(aggregate_id, sequence) enforces ordering long seq = order.getVersion(); for (DomainEvent event : newEvents) { eventStore.append(order.getId(), ++seq, event); } // Create snapshot every 50 events if (seq % 50 == 0) { snapshotRepo.save(new AggregateSnapshot(order.getId(), seq, serialize(order))); } }
```

3. Idempotency Key – Server-Side Deduplication

```
-- processed_requests table CREATE TABLE processed_requests ( idempotency_key VARCHAR(64) PRIMARY KEY, status_code INT NOT NULL, response_body TEXT NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), expires_at TIMESTAMPTZ NOT NULL ); @Component public class IdempotencyInterceptor implements HandlerInterceptor { @Override public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) throws Exception { String key = req.getHeader("X-Idempotency-Key"); if (key == null) return true; // idempotency key optional for GET // Lock row to prevent concurrent processing with same key Optional<ProcessedRequest> existing = processedRequestRepo.findByKeyWithLock(key); // SELECT ... WHERE idempotency_key=? FOR UPDATE SKIP LOCKED if (existing.isPresent()) { ProcessedRequest rec = existing.get(); res.setStatus(rec.getStatusCode()); res.setContentType("application/json"); res.getWriter().write(rec.getResponseBody()); return false; // short-circuit – do not invoke controller } // Mark as IN_PROGRESS (prevents concurrent duplicate processing) processedRequestRepo.insertInProgress(key, Instant.now().plus(Duration.ofHours(24))); req.setAttribute("IDEMPOTENCY_KEY", key); return true; } @Override public void afterCompletion(HttpServletRequest req, HttpServletResponse res, Object handler, Exception ex) { String key = (String) req.getAttribute("IDEMPOTENCY_KEY"); if (key != null && ex == null) { // Persist final response for future duplicates String body = ((ContentCachingResponseWrapper) res).getBodyAsString(); processedRequestRepo.updateCompleted(key, res.getStatus(), body); } }
```

4. Transactional Outbox Pattern with Spring

```
-- outbox table CREATE TABLE outbox_events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), event_type VARCHAR(100) NOT NULL, aggregate_id VARCHAR(100) NOT NULL, payload JSONB NOT NULL, published BOOLEAN NOT NULL DEFAULT false, created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); @Service @Transactional public class OrderService { public Order createOrder(CreateOrderCommand cmd) { // Business logic Order order = new Order(cmd); orderRepo.save(order); // Write event to outbox IN THE SAME TRANSACTION OutboxEvent outboxEvent = OutboxEvent.builder().eventType("OrderCreated").aggregateId(order.getId()).payload(objectMapper.valueToTree(new OrderCreatedEvent(order))).build(); outboxRepo.save(outboxEvent); // If TX rolls back → outbox row is also rolled back (atomic!) return order; } } // Relay: scheduled poller (or Debezium CDC on outbox table)
```

```
@Scheduled(fixedDelay = 1000) @Transactional public void publishOutboxEvents() {
    List<OutboxEvent> unpublished =
    outboxRepo.findByPublishedFalseOrderByCreatedAt(PageRequest.of(0, 100)); for (OutboxEvent
    event : unpublished) { kafkaTemplate.send(event.getEventType(), event.getAggregateId(),
    event.getPayload().toString()); event.setPublished(true); outboxRepo.save(event); // Mark
    published AFTER successful Kafka send // If crash between send and mark: Kafka gets duplicate
    → consumer deduplicates } }
```

5. Inbox Deduplication Pattern – Idempotent Kafka Consumer

```
-- processed_events table (inbox) CREATE TABLE processed_events ( event_id VARCHAR(100) PRIMARY
KEY, -- Kafka partition:offset or event UUID event_type VARCHAR(100), processed_at TIMESTAMPTZ
DEFAULT now() ); @KafkaListener(topics = "order-events", groupId = "inventory-service")
@Transactional public void handleOrderEvent(ConsumerRecord<String, String> record) { // Derive
stable event ID from Kafka coordinates String eventId = record.topic() + ":" +
record.partition() + ":" + record.offset(); // Inbox check – within the SAME transaction as
business logic if (processedEventRepo.existsById(eventId)) { log.debug("Duplicate event {}
skipped", eventId); return; // idempotent: already processed } // Parse and process OrderEvent
event = objectMapper.readValue(record.value(), OrderEvent.class);
inventoryService.reserveStock(event.getOrderid(), event.getItems()); // Mark as processed IN
SAME TX as inventory update processedEventRepo.save(new ProcessedEvent(eventId,
event.getType())); // If TX commits: both inventory update and processed marker saved
atomically // If TX rolls back: neither saved → safe to retry → won't double-process } //
Consumer config for exactly-once with Kafka transactions: // isolation.level=read_committed
(skip messages from aborted/in-progress transactions) // enable.auto.commit=false (manual
offset commit) // Spring Kafka: ContainerProperties.AckMode.MANUAL_IMMEDIATE
```

6. @TransactionalEventListener and Event Publishing

```
// @TransactionalEventListener fires AFTER the DB transaction commits // Prevents publishing
events that were later rolled back @Service @Transactional public class PaymentService {
@Autowired private ApplicationEventPublisher eventPublisher; public void processPayment(String
orderId, BigDecimal amount) { Payment payment = new Payment(orderId, amount,
PaymentStatus.SUCCESS); paymentRepo.save(payment); // Publish domain event – listener fires
AFTER this TX commits eventPublisher.publishEvent(new PaymentProcessedEvent(orderId, amount));
// If TX rolls back after this line → listener does NOT fire } } @Component public class
PaymentEventListener { @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onPaymentProcessed(PaymentProcessedEvent event) { // This fires only after the
enclosing TX has committed // Safe to publish to Kafka / update outbox here
kafkaTemplate.send("payments", event.getOrderid(), objectMapper.writeValueAsString(event)); //
Note: if Kafka send fails here, the DB TX is already committed // → use outbox pattern for
guaranteed delivery instead } @TransactionalEventListener(phase =
TransactionPhase.AFTER_ROLLBACK) public void onPaymentRolledBack(PaymentProcessedEvent event)
{ log.warn("Payment TX rolled back for order {}", event.getOrderid()); // Compensating action
if needed } // TransactionPhase options: // BEFORE_COMMIT – fires before commit (still within
TX, can roll back) // AFTER_COMMIT – fires after successful commit (most common) //
AFTER_ROLLBACK – fires after rollback // AFTER_COMPLETION – fires after either commit or
rollback }
```

Interview Questions & Answers

Q1. What is Event Sourcing and how does it differ from traditional CRUD?

In traditional CRUD, you overwrite the current state: `UPDATE orders SET status='SHIPPED'`. The previous state is lost. In Event Sourcing, you append an immutable event to the event log: `INSERT domain_events(aggregate_id='order-1', event_type='OrderShipped', ...)`. The current state is derived by replaying all events from the beginning. Benefits: complete audit trail (every state transition recorded), temporal queries (what was the state at time T?), event-driven integration (other services subscribe to events), easy debugging (reproduce any past state by replaying). Drawbacks: eventual consistency for reads (read model may lag), replay performance (mitigated by snapshots), schema evolution complexity (handling old event formats), steeper learning curve. Event Sourcing pairs naturally with CQRS: the write side appends events, the read side maintains denormalized projections.

Q2. Describe the event store schema and why the sequence number matters.

Event store schema: `aggregate_id` (identifies which aggregate the event belongs to), `sequence` (monotonically increasing per aggregate — event 1, 2, 3...), `event_type` (e.g., `OrderCreated`, `ItemAdded`), `payload` (JSON/JSONB with event data), `metadata` (`correlation_id`, `causation_id`, `user_id` for tracing), `occurred_at` (timestamp). The sequence number serves two purposes: (1) Ordering — events must be replayed in sequence order to reconstruct state correctly. Out-of-order replay produces wrong state. (2) Optimistic concurrency — the event store has a `UNIQUE` constraint on (`aggregate_id`, `sequence`). When two concurrent commands try to append event #5 for the same aggregate, only one succeeds; the other gets a unique constraint violation and must retry after reloading the aggregate. This is optimistic concurrency on the event log itself.

Q3. What is a projection and how do you rebuild one?

A projection (read model) is a denormalized, query-optimized view of aggregate state built by consuming events. Example: an `OrderSummaryProjection` listens to `OrderCreated`, `ItemAdded`, `OrderShipped` events and maintains a `orders_summary` table with precomputed fields (total items, total price, current status). Projections are: (1) Separated from write model — can be rebuilt at any time. (2) Eventually consistent — they may lag behind the event log. (3) Technology-independent — same events can build an Elasticsearch index, a Redis cache, or a relational table. Rebuilding: drop the projection table, replay all events from `seq=1` through the projection handler, update the projection. This allows you to retroactively add new projections without changing business logic and to fix bugs in projection logic by rebuilding.

Q4. What is a snapshot and when should you use it?

A snapshot is a cached serialization of an aggregate's state at a particular sequence number, stored alongside the event log. When loading an aggregate: (1) Load the latest snapshot (if any) — restores state to `snapshot.sequence`. (2) Load only events with `sequence > snapshot.sequence`. (3) Replay only those events. Without snapshots, loading an aggregate with 10,000 events requires loading and replaying all 10,000 — expensive. With a snapshot at `seq=9,950`, only 50 events need replaying. When to use snapshots: when aggregate event count grows beyond ~100-200 events. Common strategy: create a snapshot every N events (e.g., every 50 or 100). Snapshot must be taken AFTER successfully appending events — never before — to avoid inconsistency if the process crashes between snapshot creation and event append.

Q5. How does an idempotency key prevent duplicate payments?

Scenario: client sends `POST /payments` with `X-Idempotency-Key: uuid-123`. Network times out. Client retries. Without idempotency: both requests are processed — double charge. With idempotency key: (1) First request arrives, key not in `processed_requests`. (2) Server inserts `IN_PROGRESS` record (unique constraint prevents concurrent duplicates). (3) Processes payment, stores result (`status_code`, `response_body`) in `processed_requests`. (4) Second request arrives with same key. (5) Finds existing completed record. (6) Returns cached response immediately without charging again. Key design: the unique constraint on the `idempotency_key` column prevents two concurrent requests with the same key from both passing the lookup (one gets a constraint violation and waits or fails). Expire keys after 24-72 hours to bound table size. Also

validate that the request body matches the original for the same key — reject body mismatches.

Q6. Explain the transactional outbox pattern and what problem it solves.

Problem: after a business transaction commits (e.g., order saved to DB), you want to publish a Kafka event. Two approaches are wrong: (1) Publish to Kafka inside the DB transaction — if Kafka is unavailable, the transaction fails for a non-DB reason. (2) Publish after commit — if the process crashes between commit and publish, the event is lost. The outbox pattern solves this: write the event to an outbox table in the SAME database transaction as the business data. Since both writes are in the same ACID transaction: if the business write commits, the outbox row commits too (event will be published eventually); if the transaction rolls back, the outbox row is also rolled back (event not published). A relay process (scheduler or Debezium CDC) reads unpublished outbox rows and publishes them to Kafka, then marks them as published. This guarantees at-least-once delivery with no lost events.

Q7. What is the inbox (deduplication) pattern and how does it complement the outbox?

The outbox pattern guarantees at-least-once delivery from DB to Kafka. This means consumers may receive duplicate messages (relay may publish before marking as published, then retry after crash). The inbox pattern makes consumers idempotent: before processing a message, check a `processed_events` table for the message's unique ID (Kafka partition:offset or a UUID from the event payload). If found: skip. If not found: process the message and insert the ID into `processed_events` — both in the same DB transaction as the business logic. Atomicity is key: if the business logic and `processed_events` insert are in the same transaction, either both commit or both roll back. This prevents the scenario of: (1) Process message. (2) Crash before inserting `processed_events`. (3) Re-deliver message. (4) Process again. The outbox + inbox combination achieves effectively-exactly-once semantics across distributed systems.

Q8. What are Kafka's exactly-once semantics (EOS) and how do transactions work?

Kafka EOS uses producer transactions to atomically publish messages to topics AND commit consumer offsets. Configuration: `enable.idempotence=true` (assigns producer ID + sequence per partition; broker deduplicates retries), `transactional.id=unique-per-producer-instance` (enables transactions). Flow: (1) `producer.initTransactions()`. (2) `producer.beginTransaction()`. (3) `producer.send()` one or more messages. (4) `producer.sendOffsetsToTransaction(offsets, groupMetadata)` — atomically commits consumer group offsets. (5) `producer.commitTransaction()` or `producer.abortTransaction()`. Consumer side: `isolation.level=read_committed` — only sees messages from committed transactions (messages from aborted transactions are invisible). This prevents partially-processed transactions from being visible to consumers. Limitation: EOS within a single Kafka cluster; cross-cluster or cross-system EOS requires additional patterns (outbox + inbox).

Q9. What does `@TransactionalEventListener` do and why is it safer than a regular `@EventListener`?

`@EventListener` fires immediately when `ApplicationEventPublisher.publishEvent()` is called — potentially inside the still-open DB transaction. If the listener publishes a Kafka event and then the DB transaction rolls back (e.g., a constraint violation later in the method), the Kafka event has already been published but the DB change is reverted. Downstream consumers see an event for a transaction that never committed. `@TransactionalEventListener` with `phase=AFTER_COMMIT` fires only after the enclosing transaction successfully commits. If the transaction rolls back, the listener never fires. This prevents phantom events. Trade-off: if the listener itself fails (Kafka is down), the DB is already committed — you cannot roll it back. For guaranteed delivery, combine `@TransactionalEventListener` with the outbox pattern: the listener writes to the outbox table (still in a new transaction post-commit), and the relay handles Kafka publishing.

Q10. How do you handle schema evolution of events in an event-sourced system?

Events are immutable — you cannot change old events in the log. Schema evolution strategies: (1) Weak schema — store events as JSONB; add new optional fields; replay code handles both old (missing field) and

new (field present) formats using null-defaults. (2) Upcasting — at load time, transform old event format to current format before passing to `apply()`. Example: `EventUpcaster.upcast(v1Event) → v2Event`. (3) Copy-and-transform — for major schema changes, replay all old events through a migration and write new events to a new aggregate (new stream). (4) Event versioning — name events explicitly: `OrderCreatedV1`, `OrderCreatedV2`; handle each version in `apply()`. Best practices: always add fields as optional (never remove or rename). Use Avro/Protobuf with a schema registry for typed schemas. Test replay of old events with every schema change.

Q11. How do projections handle eventual consistency and what compensating patterns exist?

Projections lag behind the event log — reads from the projection DB may show stale data immediately after a write. Patterns to manage this: (1) Read-your-writes: after a command, return the result of applying the events directly (from the write model), not from the projection. (2) Version tracking: include the event sequence in the query response; clients poll until the projection reaches the expected sequence. (3) Optimistic UI: update the UI immediately based on the command's expected outcome, then reconcile with the server when the projection catches up. (4) Causally consistent reads: route reads for an aggregate to the same projection instance that processed its last event (sticky sessions or consistent hashing). (5) Accept eventual consistency: design the business domain to tolerate brief stale reads — most systems can.

Q12. Describe a complete CQRS + Event Sourcing flow for a hotel booking system.

Write side (Commands): `BookRoom` command → `BookingCommandHandler` loads `Booking` aggregate via event replay → validates business rules (room available, dates valid) → appends `RoomBooked` event to event store (with sequence optimistic concurrency). Read side (Projections): `BookingProjector` subscribes to `RoomBooked` events (via Kafka or DB poll) → updates `room_availability_view` and `booking_summary_view` tables. Query: `GET /rooms/available` reads from `room_availability_view` (fast, denormalized). Idempotency: `BookRoom` command carries idempotency key → command handler checks `processed_commands` table before appending event. Outbox: `RoomBooked` event also written to outbox table in same TX as event store append → relay publishes to Kafka for downstream services (email, billing). Inbox: billing service deduplicates `RoomBooked` events before charging. This full flow achieves: audit trail (event log), scalable reads (projections), loose coupling (Kafka), idempotent processing (inbox/outbox).

Q13. What is the difference between a domain event and an integration event?

Domain event: an immutable fact that happened within a bounded context, expressed in domain language (`OrderConfirmed`, `PaymentFailed`). Used internally within the service to trigger side effects and build projections. Published inside the service's domain layer. Integration event: an event published across bounded context boundaries (to Kafka, `EventBridge`). Often a simplified, versioned contract — strips internal domain details, contains only what downstream consumers need. Example: `OrderShippedIntegrationEvent` contains `orderId`, `customerId`, `shippedAt` — not internal `Order` aggregate fields. Best practice: translate domain events to integration events at the anti-corruption layer (outbox/relay). This prevents internal domain changes from breaking downstream consumers. Domain events may change frequently; integration events should be stable and versioned (v1, v2) with backward compatibility maintained.

Q14. How do you implement a saga pattern with event sourcing?

A saga is a long-running distributed transaction composed of multiple local transactions with compensating actions. With event sourcing: (1) Choreography saga — each service listens to events and emits events. No central coordinator. `OrderCreated` → inventory reserves stock (`StockReserved`) → payment charges (`PaymentCharged`) → order confirmed. If payment fails: `PaymentFailed` event → inventory releases stock (compensating event). (2) Orchestration saga — a `SagaOrchestrator` aggregate in the event store coordinates: appends `BookFlightRequested`, `HotelBookingRequested` as events; listens for replies; appends compensating events on failure (`CancelFlightRequested`). Event sourcing benefits for sagas: the saga state is the replay of its events — fully auditable, replayable, and debuggable. Idempotency keys on compensating

commands prevent double-cancellation.

Q15. What monitoring and operational concerns are specific to event-sourced systems?

Key metrics: (1) Event store size — events per aggregate (flag aggregates with >500 events as needing snapshot). (2) Projection lag — how far behind is each projection from the latest event? Alert if lag > N seconds. (3) Replay time — how long does a full replay take? This limits the RTO for rebuilding projections. (4) Outbox table backlog — unpublished outbox rows accumulating indicates relay failure; alert if count > threshold. (5) Idempotency key table size — ensure cleanup job runs. Operational concerns: (a) Schema migration strategy for events (upcasting). (b) Snapshot strategy and snapshot cleanup (delete old snapshots). (c) Event log archival for old aggregates (move to cold storage). (d) Replay performance — test full replay time in staging before production. (e) Causality tracking — use correlation_id and causation_id in event metadata for distributed tracing.

Q16. How do you test an event-sourced application?

Testing layers: (1) Aggregate unit tests — given existing events (history), when command applied, then new events emitted. Pure function test, no DB needed. Example: given([OrderCreated, ItemAdded]), when(ShipOrder), then([OrderShipped]). (2) Projection tests — given events, assert projection state. Example: given([OrderCreated, PaymentReceived]), assert orderSummaryView.status=PAID. (3) Saga tests — given sequence of events, assert saga emits correct commands and compensating actions on failure. (4) Integration tests — use TestContainers with real PostgreSQL and Kafka; test full command → event → projection pipeline. (5) Replay tests — load 1000 historical events, replay, verify final state matches expected. (6) Idempotency tests — send same command twice with same idempotency key, assert exactly one event appended. These tests are fast and reliable because the aggregate apply() is a pure function.

Q17. What are the trade-offs of using event sourcing vs a traditional audit log?

Traditional audit log: a separate audit table (INSERT INTO audit_log(entity, action, old_value, new_value, user, timestamp)) added alongside the main state table. Pros: simple to add to existing CRUD app; main queries against state table are unchanged; no replay needed. Cons: audit log is supplementary — source of truth is still the state table; state can be changed without an audit log entry (bug or direct DB access); rebuilding state from audit log is complex and often not implemented. Event Sourcing: events ARE the source of truth — state is derived from events. Pros: complete, tamper-evident history; state can always be rebuilt; event-driven integration is natural; temporal queries possible. Cons: complex infrastructure; eventual consistency for reads; schema evolution complexity; performance depends on snapshot strategy. Choose event sourcing when: the audit trail is a core business requirement, the domain is complex with many state transitions, or event-driven integration is central to the architecture.

Q18. How do you handle concurrency in event sourcing without transactions across services?

Each aggregate is its own consistency boundary. Concurrency within a single aggregate: use the sequence number as an optimistic lock. Two commands try to append event #5 for aggregate order-1 → unique constraint (aggregate_id, sequence) ensures only one succeeds → the other gets a conflict, reloads the aggregate, re-evaluates business rules, retries. Cross-aggregate concurrency: in event sourcing, you cannot span a single transaction across multiple aggregates. Approach: (1) Design aggregate boundaries to contain all state needed for an invariant within one aggregate. (2) Use sagas for multi-aggregate workflows with compensating events. (3) Accept eventual consistency between aggregates — invariants that span aggregates are enforced eventually. (4) Use process managers (saga orchestrators) to coordinate multi-step workflows with idempotent compensating commands.

Q19. What is the difference between event sourcing and Kafka event streaming?

Event Sourcing: an architectural pattern for the write side of a domain — the event log IS the database. Events are stored in an event store (often a relational DB or EventStoreDB) with strict per-aggregate ordering

and optimistic concurrency. Purpose: derive aggregate state, provide audit trail, enable CQRS. Events are typically kept forever. Kafka Event Streaming: a distributed messaging system for decoupled, scalable event distribution. Topics have configurable retention (days/bytes), not meant for infinite storage. Purpose: publish-subscribe, stream processing, inter-service communication. Kafka is not an event store — you cannot efficiently query 'all events for order-123'. They complement each other: use an event store for aggregate state (event sourcing), use Kafka to distribute events to other services (integration events), using the outbox pattern to bridge the two reliably.

Q20. How would you migrate a traditional CRUD system to event sourcing incrementally?

Incremental migration strategies: (1) Strangler Fig — add event sourcing to new aggregates or new bounded contexts while keeping existing CRUD for legacy parts. (2) Event log alongside CRUD — continue writing to the state table AND append events to an event log. The event log is supplementary at first; gradually shift source of truth to events. (3) CDC bootstrapping — use Change Data Capture (Debezium) to capture all historical changes from the existing DB and replay them as events to bootstrap the event store. (4) Dual-write period — write to both old DB and event store; run both projections; compare output for correctness. (5) Cut-over — once confidence is established, stop writing to old DB; serve reads exclusively from event-sourced projections. Key risks: event schema design for legacy data, historical data volume (many aggregates × many events), team knowledge curve.